Frontend

Next.js 16 App Router, React 19, Tailwind CSS 4, Animate UI over Radix. Everything renders on the server and reads live data; there is no client-side fetching layer.

The running app documents its own parts at `/styleguide` — colour, typography, every component, live rather than as code listings.

The idea

A screen should not invent primitives. It composes them, and it declares its data.

- **What a record *is*** is one `FieldSpec[]`. The client renders that list and the server action reads the submission back out of the same list, so adding a column is one line rather than an edit in three files.
- **What a list *shows*** is one `Column[]` plus a `FilterBar` declaration.
- **Everything else** — spacing, the busy state, where an error message lands, what a delete confirms — belongs to the shared components and is therefore identical everywhere.

Directories

Path	Holds
`components/ui/`	The library. Import from `@/components/ui`.
`components/data/`	Reading: `DataTable`, `FilterBar`, `StatusBadge`, `EmptyState`, `Section`, `StatTile` / `StatGrid`, `PersonCell`, skeletons.
`components/form/`	Writing: `RecordDialog`, `RecordForm`, `RowActions`, `ActionButton`.
`components/app/`	The shell: `AppSidebar`, `AppBreadcrumbs`, `Notifications`.
`components/documents/`	`DocumentDetail`, `SignPanel`, `SignaturePad`, `SubmitPanel` — shared by the HR and employee views of the same document.
`components/employees/`	`AvatarPicker`.
`lib/`	`api-client`, `session`, `access`, `fields`, `mutate`, `form-state`, `refs`, `format`, `status`, `avatar`, `paged`.
`app/(app)/<domain>/`	One folder per screen.
`app/(me)/me/`	The employee's own space: punch card, attendance, leave, pay, documents.
`app/api/*/route.ts`	Proxies that attach the session cookie to a request the browser cannot authenticate itself — payslip PDFs, document files, avatars, the company logo, and the notification stream.

Adding a screen, end to end

Four small files. Employees is the worked reference — read `app/(app)/employees/` alongside this.

1. `fields.ts` — what the record is

A plain module: no `"use server"`, no `"use client"`. It exports a function taking the loaded reference lists, because the page has them and the server action does not need them.

```
export function contractFields(refs?: Partial<Refs>): FieldSpec[] {
  return [
    { name: "name", label: "Name", required: true },
    { name: "employeeId", label: "Employee", type: "select",
      options: refs?.employees, required: true, createOnly: true },
    { name: "dateStart", label: "Start date", type: "date", required: true },
    { name: "dateEnd", label: "End date", type: "date", clearable: true },
    { name: "wage", label: "Wage", type: "number", min: 0, required: true },
    { name: "status", label: "Status", type: "select",
      options: statusOptions(CONTRACT_STATUSES) },
    { name: "notes", label: "Notes", type: "textarea", clearable: true },
  ];
}
```

Field specs cross the server/client boundary as props, so they must stay **fully serializable** — plain data, never a function.

Property	Meaning
`type`	`text` · `email` · `tel` · `password` · `number` · `date` · `datetime` · `textarea` · `select` · `multiselect` · `switch` · `color` · `json`
`required`	Checked before the request leaves the browser. Set it only where the API requires it.
`clearable`	Submitting empty sends `null`, which clears the column. Without it an empty optional field is omitted, so a PATCH leaves the stored value alone. **Never on an enum** — the API rejects null there.
`createOnly`	Hidden on the edit form. For a column the API will not update, or a password.
`defaultValue`	Seeds a create. An edit ignores it; the record wins. Set it wherever the API has a default of its own.
`span`	`"full"` to take both grid columns. Textareas always do.
`hint`	A line under the control, for what the label cannot carry.
`json`	The form renders no control; a bespoke one supplied through `extras` fills a hidden input. See the working-schedule week editor in `app/(app)/settings/_components/`.

2. `actions.ts` — how it is written

```
"use server";

const CONTRACT = { path: "/contracts", fields: contractFields(), label: "Contract" };

/** Creates when the form carries no id, updates when it does. */
export async function saveContract(_previous: FormState, formData: FormData) {
  return saveRecord(CONTRACT, formData);
}

export async function deleteContract(id: string) {
  return deleteRecord(CONTRACT, id);
}
```

For a verb the API owns rather than a record edit:

```
export async function sendPayslips(id: string) {
  return callAction<SendPayslipsResultDto>({
    path: `/payruns/${id}/send-payslips`,
    message: (r) => `${r.sent} sent${r.failed ? `, ${r.failed} failed` : ""}.`,
  });
}
```

`callAction` takes a message **function** where the API reports what it actually did, so the confirmation carries it instead of a fixed string.

3. `page.tsx` — the screen

```
const session = await requireAccess("contracts");
const [rows, refs] = await Promise.all([
  apiFetch<ContractDto[]>("/contracts", { query: { q, status } }),
  loadRefs(["employees", "positions", "schedules", "structures"]),
]);
```

`loadRefs` fetches the option lists relation selects need, in parallel, failing soft so a role that cannot read a list still gets the page.

Then the create control goes in the filter bar's `actions` slot:

```
<FilterBar
  search={{ placeholder: "Search contract or employee" }}
  selects={[{ key: "status", options: statusOptions(CONTRACT_STATUSES) }]}
  quickFilters={[{ key: "expiring", value: "true", label: "Expiring soon" }]}
  count={{ total: rows.length, noun: "contract" }}
  actions={canCreate ? (
    <RecordDialog title="New contract" fields={contractFields(refs)}
                  action={saveContract} submitLabel="Create contract" />
  ) : null}
/>
```

and the row menu goes in a trailing column:

```
<RowActions
  edit={{ title: "Edit contract", fields: contractFields(refs),
          action: saveContract, record: row }}
  remove={{ action: deleteContract.bind(null, row.id),
            title: `Delete ${row.name}?`, description: "…" }}
/>
```

A server action passed into a client component **must** be bound with `.bind(null, id)`. A closure is not serializable across that boundary.

4. Gate everything

`can(session.role, "contracts", "create" | "update" | "delete")`. A role without the permission must not see the control.

Rules worth knowing

Offer Edit only where the page has the whole record. An edit form submits every field it renders, and a `clearable` field left blank is sent as `null`. Wiring an edit dialog to a summary DTO would silently wipe the columns the summary omits. Employees is the case in point: the list menu offers only Delete, and Edit lives on the record page where `EmployeeDetailDto` is loaded.

List pages carry no heading block. The breadcrumb already says where you are. Detail pages keep a header, because it carries the record's identity.

Filters live in the URL, so a filtered view is a link. `FilterBar` writes them; the page reads `searchParams` and passes them to the API. Filtering happens server-side, so the browser never holds the full table and the Employee role's scoping is applied at the source.

Every destructive action confirms, and the confirmation names the consequence — "Delete workspace", not "OK". `RowActions.remove` does this; `ActionButton` takes a `confirm` prop.

Import `FormState` and `FORM_IDLE` from `lib/form-state`, not `lib/mutate`. `mutate.ts` is `server-only` and reaches `next/headers`, so a client component importing the idle value from it drags the whole server client into the browser bundle. The *type* is erased at compile time and would be harmless; `FORM_IDLE` is a real value, and that is what the split exists for.

A file the browser cannot authenticate goes through a proxy route. The JWT is httpOnly, so neither an `<img src>` nor a plain link can carry it. `app/api/…/route.ts` reads the cookie server-side and streams the bytes back. Avatar URLs put the file id on the query string (`avatarUrl` in `lib/avatar.ts`) so a replaced picture is a new URL rather than a cached one.

`StatTile` **is the same component on nine screens**, so it stays the same size on all of them — change it once and every screen moves together. `StatGrid` takes a `columns` count, `4` or `5`; five is for a row with one more thing worth saying than it has room for at four.

`suppressHydrationWarning` **has to sit on the element holding the differing text**, not an ancestor of it. Put it on a wrapper and React still compares the text inside and reports the mismatch. This bites on anything rendered from the reader's own clock — a date drawn on the server in one timezone and rehydrated in another — where a re-render to reconcile is not worth the cost.

How errors reach the user

`class-validator` returns messages that lead with the property name. `toFieldErrors` in `lib/mutate.ts` re-attributes each one to the field it belongs to, reworded with the label the form actually shows — so "wage must not be less than 0" appears under **Wage** as "Wage must not be less than 0", not as a banner full of internal names. Anything it cannot attribute becomes the banner.

Success is a toast, but only where the change leaves no visible trace. A row that updates in place in front of you does not need one.

Theming

Light and dark, in `components/ui/theme.tsx`. It is deliberately local rather than a library: the libraries that do this render their anti-flash script from inside a client component, which React 19 refuses to execute and then regenerates the whole tree over. Here the script is rendered by the server layout into the document head, where it runs before first paint and never enters the React tree.

The palette is CSS custom properties in `app/globals.css` — a `:root` block and a `.dark` block. Components reference tokens (`bg-card` , `text-muted-foreground`), never raw colours.

Shell states

File	When
`app/(app)/loading.tsx`	Any screen without its own. A filter bar over a list, the shape nearly all of them have.
`app/(app)/employees/loading.tsx`	Employees, whose card grid would otherwise reflow.
`app/(app)/error.tsx`	A screen threw — usually the API being unreachable. Offers a retry and quotes the message.
`app/(app)/not-found.tsx`	`notFound()` on a detail route: a stale link, or a record since removed.

Conventions

- Import from the barrel: `import { Button, Field, Input } from "@/components/ui"` .
- Icons go on a button as `startIcon` / `endIcon` , never as children, which keeps them on the flex baseline.

- Naming is minimal: `saveX`, `deleteX`, `verbX`.
- Comments say **why**, not what.
- Sentence case everywhere. No marketing tone.

> **Do not run** `shadcn add -o` for anything depending on the registry's own `button` item. It overwrites `components/ui/button.tsx`, which is a different component that happens to share a name. It has happened twice.